

FitCoach AI

Dossier de projet — Bloc 2

Concevoir et développer une solution applicative

RNCP 39583 — Expert en Développement Logiciel

Candidat	Alley Eddine (Alleycom)
Commanditaire	Markus — coach sportif privé
Projet	FitCoach AI — plateforme de coaching sportif (coach / élève / autonome)
Épreuve	Bloc 2 — rendu écrit : code source + dossier
Front en production	https://fitapp-ai-ten.vercel.app (Vercel)
Back en production	5 microservices Fastify sur VPS Hetzner (IP 188.245.65.113), orchestrés par Coolify
Service auth	https://yc4cw0cowcsw0ws4gcosc.188.245.65.113.sslip.io (vérifiable via /health)
Service api	https://g8ks4w0o80g04o4kggo8csoo.188.245.65.113.sslip.io (vérifiable via /health)
Service payment	https://t88cs0gswco8s80ggo44kc8c.188.245.65.113.sslip.io (vérifiable via /api/payment/plans)
Version du dossier	1.0 — 22 juillet 2026

Table des matières

1. Introduction et périmètre du dossier	4
2. Le produit en production : trois parcours, cinq services	5
3. Environnements de déploiement, de test et suivi de la performance — C2.1.1.....	6
3.1 Un monorepo, trois environnements distincts.....	6
3.2 Suivi de la performance et des ressources : Prometheus / Grafana	7
3.3 Incident réel : saturation mémoire du VPS de production	7
3.4 Indicateurs de suivi retenus.....	8
4. Intégration continue — C2.1.2.....	9
4.1 Le pipeline <code>ci.yml</code>	9
4.2 La CI comme garde-fou de fusion	9
4.3 Ce que la CI ne fait pas encore (limite assumée)	9
5. Prototype : construire une base sécurisée dès l'amorçage — C2.2.1	11
5.1 Un prototype qui est aussi la base de la production	11
5.2 Sécurité intégrée dès l'amorçage.....	11
5.3 Mobile et web : un choix assumé, pas un renoncement	12
5.4 Preuve d'exécution : comptes de démonstration.....	12
6. Tests unitaires — C2.2.2	13
6.1 Stratégie : isoler la logique métier de l'infrastructure	13
6.2 Exemples représentatifs	13
6.3 Inventaire des tests.....	14
6.4 Limite assumée et axe d'amélioration	15
7. Développer une solution évolutive, sécurisée et accessible — C2.2.3.....	16
7.1 Évolutivité : un cadre de développement documenté et arbitré	16
7.2 Sécurité : une déclinaison opérationnelle d'OWASP	17
7.3 Un exemple complet de logique sécurisée et transactionnelle.....	18
7.4 Accessibilité.....	18
7.5 Un cycle de développement complet, de bout en bout	18
8. Déploiement continu — C2.2.4.....	20
8.1 Vue d'ensemble de la chaîne	20
8.2 Le VPS ne construit rien	20
8.3 Étanchéité public/privé au niveau du déploiement.....	21
8.4 Incidents de mise en production, diagnostic et résolution	21
8.5 Discipline Git derrière le déploiement continu.....	23
8.6 Indicateurs de déploiement.....	24
9. Cahier de recettes — C2.3.1	25
9.1 Méthode	25

9.2 Scénarios — parcours autonome.....	25
9.3 Scénarios — parcours coach	25
9.4 Scénarios — parcours élève.....	26
9.5 Scénarios transverses (sécurité, notifications)	27
9.6 Synthèse du cahier de recettes.....	28
10. Plan de correction des bugs — C2.3.2	29
10.1 Processus de gestion des anomalies.....	29
10.2 Anomalies résolues (preuve par le code et l'historique Git)	29
10.3 Anomalies ouvertes — détaillées	30
10.4 Priorisation.....	31
10.5 Indicateurs de suivi des anomalies	31
11. Documentation technique : déploiement et utilisateur — C2.4.1	33
11.1 Documentation de déploiement.....	33
11.2 Documentation de décision architecturale	33
11.3 Documentation portée par le code lui-même	33
11.4 Documentation utilisateur : état actuel et plan	33
12. Synthèse et chemin d'évolution.....	35
13. Annexes.....	36
13.1 Artefacts publics cités dans ce dossier	36
13.2 Comptes de démonstration	36
13.3 Issues et pull requests citées	36
13.4 Glossaire.....	36

1. Introduction et périmètre du dossier

Le Bloc 1 (oral, cadrage) a établi le besoin, le commanditaire réel — Markus, coach sportif privé qui me suit personnellement et gère aujourd'hui ses programmes par PDF envoyés par email et son suivi par WhatsApp —, l'étude comparative d'architecture (microservices retenus face au monolithe et au BaaS) et la modélisation de la base. Ces éléments ne sont pas repris ici : on s'y réfère (`docs/rncp/bloc1/Bloc1_Deck.md`, `docs/rncp/bloc1/Bloc1_Script_Soutenance.md`) chaque fois que c'est utile à la compréhension d'un choix de développement.

Le Bloc 2 documente **comment le cadrage a été transformé en un logiciel qui tourne réellement en production**, avec la rigueur d'ingénierie attendue au niveau Expert : environnements séparés, intégration et déploiement continus, tests automatisés, sécurité intégrée au développement (et non ajoutée après coup), gestion tracée des anomalies, documentation technique.

Méthode de preuve. Chaque affirmation de ce dossier est adossée à un artefact vérifiable du dépôt : un chemin de fichier, un numéro d'issue ou de pull request GitHub, un nom de branche, un extrait de code réel. Rien n'est présenté comme acquis sans preuve traçable — c'est la même exigence de traçabilité que celle appliquée aux données du produit lui-même (journalisation des notifications, historique Git, migrations versionnées).

Structure. Chaque chapitre annonce en en-tête la ou les compétences C2.x.x qu'il couvre, dans l'ordre du référentiel : environnements et CI (C2.1.1, C2.1.2), développement du prototype à la solution évolutive et sécurisée avec ses tests et son déploiement continu (C2.2.1 à C2.2.4), recette et correction des anomalies (C2.3.1, C2.3.2), documentation (C2.4.1).

2. Le produit en production : trois parcours, cinq services

FitCoach AI sert aujourd'hui trois parcours réels, avec des comptes de démonstration actifs (`markus.demo@fitcoach.local` pour le rôle coach, `emma.demo@fitcoach.local` pour le rôle élève) :

Parcours	Rôle applicatif	Ce qu'il fait	Support
Autonome	<code>user</code>	Self-tracking (poids, pas, séances), calcul du besoin calorique (Mifflin-St Jeor), Frigo Mode : génération de recettes par IA à partir de ce qu'il a chez lui	PWA installable
Coach	<code>coach</code>	Crée des programmes d'entraînement versionnés par phase, des plans nutrition (repas par kcal/macros + compléments), invite des élèves, suit leur adhérence	Desktop
Élève	<code>student</code>	Reçoit un programme et un plan nutrition déjà réglés, exécute sans configurer, voit ses recettes IA contraintes par le cadre du coach	PWA mobile

Le front (Next.js 15, App Router, PWA) est déployé sur Vercel à l'URL <https://fitapp-ai-ten.vercel.app>. Il consomme trois services backend exposés publiquement — `auth`, `api`, `payment` — via des URLs `sslip.io` générées par Coolify sur l'IP du VPS (188.245.65.113), avec certificat TLS Let's Encrypt automatique (URLs récapitulées en tête de dossier) (le détail du schéma de déploiement fait l'objet du chapitre 8). Deux services, `ai` et `notifications`, ne sont volontairement **jamais exposés** publiquement : ils ne répondent que sur le réseau privé Coolify, appelés uniquement par les autres services via l'en-tête `x-internal-key`.

Ce chapitre pose le décor ; les chapitres suivants détaillent, compétence par compétence, comment cet état de production a été obtenu et est maintenu.

3. Environnements de déploiement, de test et suivi de la performance — C2.1.1

Compétence couverte : C2.1.1 — Mettre en place les environnements de déploiement et de test nécessaires, en assurer le suivi et la performance.

3.1 Un monorepo, trois environnements distincts

Le projet est un monorepo **pnpm workspaces** (`pnpm-workspace.yaml`), avec cinq services Fastify 5 (TypeScript, ESM), une application front Next.js 15 et un package partagé `@fitapp/shared` (schémas Zod et types). TypeScript est utilisé en mode strict (`strict: true`, aucun `any` toléré) et en mode « solution » (`tsc -b` avec `project references`), ce qui garantit qu'un changement de type dans `packages/shared` fait immédiatement échouer la compilation de tout service qui en dépend — un garde-fou de cohérence à l'échelle du monorepo, avant même l'exécution des tests.

Trois environnements sont clairement séparés, chacun avec son fichier de configuration Docker Compose dédié :

Environnement	Fichier	Caractéristiques
Développement	<code>docker-compose.yml</code>	PostgreSQL 16 et Redis 7 exposés sur <code>localhost</code> (ports 5432/6379), schéma injecté par <code>bind-mount</code> (<code>./scripts/init-db.sql</code>), Prometheus/Grafana optionnels via le profil Compose <code>monitoring</code>
Test (CI)	exécution <code>pnpm test</code> par service, sans dépendance externe	La logique métier testée (voir chapitre 6) n'importe aucune infrastructure — aucune base de données n'est nécessaire pour faire tourner la suite de tests
Production	<code>docker-compose.prod.yml</code>	Images pré-construites tirées de GHCR, schéma PostgreSQL <i>baké dans l'image</i> (<code>services/postgres/Dockerfile</code>), services internes non exposés, healthchecks HTTP sur <code>/health</code>

En développement, chaque service se lance avec `tsx watch src/index.ts` (rechargement à chaud) et lit son propre `.env` (gitignored), validé au démarrage par un schéma Zod (`config/env.ts`). Exemple réel, service `api` :

```
const envSchema = z.object({
  PORT: z.coerce.number().default(3002),
  NODE_ENV: z.enum(['development', 'production', 'test']).default('development'),
  DATABASE_URL: z.string().url(),
  JWT_SECRET: z.string().min(32),
  INTERNAL_API_KEY: z.string().min(16).optional(),
});

const parsed = envSchema.safeParse(process.env);
if (!parsed.success) {
  console.error('Invalid environment variables:', parsed.error.flatten().fieldErrors);
  process.exit(1);
}
export const env = parsed.data;
```

Ce choix — échouer immédiatement et bruyamment plutôt que démarrer avec une configuration incomplète — est délibéré : il transforme une variable d'environnement manquante en un signal explicite dans les logs de démarrage plutôt qu'en un bug silencieux découvert par un utilisateur. C'est ce même mécanisme qui a rendu visible, en production, l'incident de variables d'environnement manquantes décrit au chapitre 8.

3.2 Suivi de la performance et des ressources : Prometheus / Grafana

Chaque service expose un point `/metrics` (bibliothèque `prom-client`), au format texte Prometheus :

```
fastify.get('/metrics', async (_request, reply) => {
  const metrics = await register.metrics();
  await reply.header('Content-Type', register.contentType).send(metrics);
});
```

`config/prometheus.yml` scrape les cinq services toutes les 15 secondes ; Grafana (port 3030) affiche un tableau de bord provisionné (« FitApp — Microservices Overview ») construit à partir de ces métriques (latence, taux d'erreur HTTP, mémoire du processus Node). Ce socle d'observabilité est ce qui a permis de **diagnostiquer**, et non simplement subir, l'incident de ressources décrit ci-dessous — un exercice concret de suivi de performance en conditions réelles, au sens de la compétence C2.1.1.

3.3 Incident réel : saturation mémoire du VPS de production

Contexte. L'hébergement backend est un VPS Hetzner à 4 Go de RAM, choisi par sobriété budgétaire (le projet est financé sur fonds propres, cf. Bloc 1 — budget). Le premier réflexe de déploiement a été de laisser Coolify construire les images Docker directement sur le VPS à chaque push, comme le permet son mode par défaut.

Symptôme. Lors d'un déploiement touchant plusieurs services en même temps, le VPS s'est figé : plus de réponse HTTP, Coolify lui-même inaccessible depuis son interface.

Diagnostic. Une fois l'accès SSH récupéré, `free -h` a montré une RAM disponible proche de zéro et un espace de **swap à 0 B** : aucune mémoire d'appoint n'existait pour absorber les pics. Cause racine : plusieurs builds Docker simultanés (compilation TypeScript, installation de dépendances pnpm pour cinq services) consomment, à eux seuls, davantage de mémoire que n'en offre un VPS d'entrée de gamme — un scénario de « voisin bruyant » auto-infligé, le processus de build assommant les conteneurs applicatifs qui tournent sur la même machine.

Correction, en deux temps :

1. **Correctif immédiat de résilience** : ajout d'un fichier d'échange (*swapfile*) de 4 Go sur le VPS, pour donner à l'hôte une marge de manœuvre mémoire en cas de pic — une mesure de mitigation, pas une solution de fond.
2. **Correctif structurel** : déplacer la construction des images **hors du VPS**, vers les runners mutualisés de GitHub Actions, et ne laisser au VPS que le rôle de **tirer** des images déjà construites (`docker pull`) — voir le détail de cette chaîne au chapitre 8 (branche `chore/deploy-prebuilt-ghcr-images`). Le VPS ne fait plus tourner que des processus `node dist/index.js` déjà compilés : sa charge mémoire redevient prévisible et bornée.

Pourquoi cette solution plutôt qu'un plus gros VPS. Augmenter la taille du VPS aurait résolu le symptôme en consommant davantage de ressources en continu (RAM allouée 24 h/24 pour un besoin — le build — qui ne dure que quelques minutes par déploiement). Déporter le build vers des runners mutualisés et déjà provisionnés par GitHub, à la demande, va dans le sens de la **sobriété** : la capacité de calcul nécessaire au build est empruntée ponctuellement à une infrastructure partagée plutôt que réservée en permanence sur un serveur dédié qui resterait sous-utilisé le reste du temps. C'est un exemple concret, appliqué au déploiement, du principe de scalabilité ciblée déjà retenu au Bloc 1 pour l'architecture applicative.

3.4 Indicateurs de suivi retenus

Indicateur	Source	Usage
Disponibilité de chaque <code>/health</code>	Healthchecks Docker Compose (prod) + <code>curl</code> manuel	Détecter un service qui ne répond plus avant l'utilisateur
Latence et taux d'erreur HTTP par service	Grafana (scrape Prometheus 15 s)	Repérer une dégradation avant l'incident
Mémoire/CPU du processus Node par service	<code>prom-client</code> (métriques par défaut collectées via <code>collectDefaultMetrics()</code>)	Anticiper une saturation (cf. §3.3)
Statut CI (vert/rouge) par branche	GitHub Actions	Indicateur de qualité en continu (détaillé chapitre 4)

4. Intégration continue — C2.1.2

Compétence couverte : C2.1.2 — Configurer un système d'intégration continue afin d'automatiser les phases de build et de test.

4.1 Le pipeline `ci.yml`

Le workflow `.github/workflows/ci.yml` s'exécute sur chaque *push* et chaque *pull request* ciblant `dev` ou `main`. Un seul job, nommé explicitement « **Lint · Typecheck · Test · Knip · Build** », enchaîne cinq contrôles :

```
concurrency:
  group: ci-${{ github.ref }}
  cancel-in-progress: true

jobs:
  quality:
    name: Lint · Typecheck · Test · Knip · Build
    steps:
      - uses: actions/checkout@v4
      - uses: pnpm/action-setup@v4          # version pilotée par "packageManager"
      - uses: actions/setup-node@v4
        with: { node-version: 20, cache: pnpm }
      - run: pnpm install --frozen-lockfile
      - run: pnpm typecheck    # tsc -b, tout le monorepo
      - run: pnpm lint        # eslint .
      - run: pnpm test        # vitest run, par service
      - run: pnpm knip        # code et dépendances mortes
      - run: pnpm build       # build de tous les workspaces
```

L'ordre est volontaire : le typecheck échoue vite et signale une erreur de contrat de données avant même de lancer les tests ; les tests valident le comportement ; **Knip** détecte le code, les fichiers et les dépendances npm devenus inutilisés dans l'ensemble des espaces de travail (`apps/web`, `services/*`, `packages/*` — configuration `knip.config.ts` à la racine) ; le build final vérifie que chaque service et le front compilent réellement en production. La clé `concurrency` avec `cancel-in-progress: true` annule automatiquement une exécution CI devenue obsolète dès qu'un nouveau push arrive sur la même branche — un choix de sobriété : pas de minutes de calcul GitHub Actions dépensées à finir de vérifier un code déjà remplacé.

4.2 La CI comme garde-fou de fusion

`pnpm install --frozen-lockfile` interdit toute dérive silencieuse du lockfile (aucune installation « au cas où » qui masquerait une dépendance non déclarée). Chaque *pull request* affiche le statut de ce job directement sur GitHub ; conformément au processus retenu (branches `feature/*` → PR vers `dev` → revue → *squash merge*), une PR dont la CI est rouge n'est pas fusionnée. Ce n'est pas une convention informelle : c'est le mécanisme qui a, par exemple, empêché la mise en échec silencieuse du typecheck lors de l'introduction du schéma partagé `saveRecipeSchema` (cycle détaillé au chapitre 7, §7.4).

4.3 Ce que la CI ne fait pas encore (limite assumée)

La CI actuelle ne construit pas et ne pousse pas les images Docker de service — ce rôle est confié à un second workflow, dédié et déclenché différemment (`build-images.yml`, détaillé au chapitre 8, C2.2.4), afin de ne pas alourdir la boucle de retour rapide attendue sur chaque PR

(typecheck/lint/test/knip/build doivent rester rapides). C'est une séparation des responsabilités délibérée entre **vérification** (`ci.yml`, sur chaque PR) et **publication d'artefact** (`build-images.yml`, seulement sur `main`).

5. Prototype : construire une base sécurisée dès l'amorçage — C2.2.1

Compétence couverte : C2.2.1 — Élaborer avec les outils appropriés un prototype de l'application, en intégrant les recommandations de sécurité, pour le mobile et le web.

5.1 Un prototype qui est aussi la base de la production

Le prototype de FitCoach AI n'est pas un artefact jetable : l'application web actuellement en production (<https://fitapp-ai-ten.vercel.app>) descend directement de la première ossature du monorepo — service `auth` avec JWT et OAuth Google, service `api` avec profils/séances/poids, calcul du besoin calorique. Le pivot vers le modèle coach/élève (rôles, programmes versionnés par phase, plans nutrition cadrés) a été construit **sur** ce socle, pas à côté. Le choix technique — PWA Next.js plutôt que deux applications natives, microservices plutôt que monolithe ou BaaS — a été argumenté au Bloc 1 (`docs/rncp/bloc1/Bloc1_Deck.md`, section « Choix de l'architecture technique ») ; ce chapitre montre comment la sécurité a été pensée **dans** ce prototype, dès son amorçage, et non ajoutée après coup.

5.2 Sécurité intégrée dès l'amorçage

Quatre mécanismes de sécurité structurent l'application depuis les tout premiers services, avant même l'ajout d'une seule fonctionnalité coach/élève :

- **Authentification par JWT signé** (bibliothèque `jose`), émis par `auth` avec un couple *issuer/audience* explicite (`fitapp:auth` / `fitapp:api`) vérifié par chaque service consommateur — un jeton émis pour un autre usage ne peut pas être rejoué ici.
- **Contrôle de rôle (RBAC)** porté par le jeton lui-même (`coach` / `student` / `user`), avec une fabrique de garde réutilisable :

```
``ts
export const requireRole =
  (...roles: UserRole[]) =>
    async (request: FastifyRequest, reply: FastifyReply) => {
      if (!request.user || !roles.includes(request.user.role)) {
        return reply.status(403).send({ error: 'Forbidden: insufficient role' });
      }
    };
``
```

- **Isolation service-à-service par clé interne** (`x-internal-key`) pour tout ce qui ne doit jamais être appelé directement par un navigateur — concrètement, le service `notifications` :

```
``ts
export const internalAuth = async (request: FastifyRequest, reply: FastifyReply) => {
  const key = request.headers['x-internal-key'];
```

```

if (typeof key !== 'string' || key !== env.INTERNAL_API_KEY) {
  return reply.status(401).send({ error: 'Invalid or missing internal API key' });
}
};
...

```

- **Validation systématique des entrées côté serveur** par des schémas Zod, jamais par confiance dans ce que le client a déjà validé (détaillé au chapitre 7).

5.3 Mobile et web : un choix assumé, pas un renoncement

Le cahier des charges évoque un volet mobile. Le prototype historique (`apps/mobile`, Expo/React Native) reste dans le dépôt à titre de référence, mais la cible produit retenue et déployée est une **PWA** unique (manifeste généré par la fonction `manifest()` de l'API Metadata de Next.js, service worker `public/sw.js`), installable sur mobile comme sur desktop, avec notifications push web (VAPID). Ce choix, argumenté au Bloc 1, évite de maintenir deux bases de code pour un besoin (coach sur grand écran, élève sur mobile) qui est avant tout un besoin d'**usage**, pas de plateforme : les deux rôles utilisent la même PWA, avec des routes et des vues différentes selon le rôle porté par le jeton.

5.4 Preuve d'exécution : comptes de démonstration

Deux comptes de démonstration permettent de rejouer le prototype en production sans donnée fictive isolée : `markus.demo@fitcoach.local` (coach, avec programme et plan nutrition réels assignés) et `emma.demo@fitcoach.local` (élève rattachée, avec historique de séances et de mensurations). Ils servent de support au cahier de recettes (chapitre 9).

6. Tests unitaires — C2.2.2

Compétence couverte : C2.2.2 — Configurer et mettre en œuvre des tests unitaires afin de valider le développement au regard des spécifications fonctionnelles et techniques.

6.1 Stratégie : isoler la logique métier de l'infrastructure

La règle appliquée dans tout le back est simple à énoncer et vérifiable dans le code : **la logique métier qui peut être pure (aucun accès réseau, base de données ou horloge système imposée) vit dans un module `domain/`, sans aucun import d'infrastructure** (pas de client `pg`, pas de client HTTP, pas de Fastify). Cette contrainte est ce qui rend la fonction unitairement testable avec Vitest, sans base de données à démarrer ni service tiers à simuler. Le code lui-même documente cette intention — extrait de `services/api/src/domain/profile-calories.ts` :

```
/**
 * Pure daily-calorie-target logic (Mifflin-St Jeor) – no infrastructure
 * imports, so it stays unit testable without a database or environment.
 */
```

Le même principe structure `services/api/src/domain/invitations.ts` (génération et expiration des codes d'invitation), `program-schedule.ts` (planification hebdomadaire), `meal-recipe-request.ts` (construction d'une requête de recette contrainte par le plan du coach) et `services/ai/src/domain/effective-tier.ts` (résolution du palier d'accès effectif). Chaque module est accompagné d'un fichier `__tests__` du même nom.

6.2 Exemples représentatifs

Calcul du besoin calorique (Mifflin-St Jeor) — la fonction centrale du parcours autonome. Le test fige l'âge en construisant une date de naissance relative à l'année courante (l'implémentation appelle `new Date()` en interne), documente cette contrainte en commentaire, puis vérifie à la fois des cas limites (âge nul ou supérieur à 120 → `null`) et des valeurs exactes calculées à la main :

```
it('computes correct BMR for a 30-year-old male, 80kg, 175cm, moderate activity, maintain', () => {
  // BMR = 10*80 + 6.25*175 - 5*30 + 5 = 1748.75 ; TDEE = 1748.75 * 1.55 = 2711
  const birthDate = new Date(`${new Date().getFullYear() - 30}-01-01`);
  expect(computeDailyCalories(makeRow({ birth_date: birthDate }))).toBe(2711);
});
```

Résolution du palier d'accès effectif — la règle métier B2B la plus sensible de la plateforme (« l'élève ne doit jamais voir le paywall B2C », cf. Bloc 1) est traduite en une fonction pure d'une ligne, et testée dans les deux sens :

```
export const resolveEffectiveTier = (
  subscription: SubscriptionTier,
  isActiveStudent: boolean
): SubscriptionTier => (isActiveStudent ? 'premium' : subscription);

it('upgrades a linked student to premium regardless of their B2C tier', () => {
  expect(resolveEffectiveTier('free', true)).toBe('premium');
});
it('keeps the B2C tier for a non-student user', () => {
  expect(resolveEffectiveTier('free', false)).toBe('free');
});
```

Cette fonction n'est pas un exercice isolé : elle est réellement appelée par `services/ai/src/routes/ai.routes.ts`, qui interroge `coach_students` pour savoir si l'utilisateur est

un élève actif avant de calculer son quota d'appels IA. La règle métier est donc **appliquée côté serveur**, pas seulement masquée côté interface.

Planification hebdomadaire — `isoDayOfWeek`, `findDayForDate`, `findNextDay` encodent la règle « lundi = 1 ... dimanche = 7 » (au lieu de la convention JavaScript native `0 = dimanche`) et la notion de jour de repos (absence de séance planifiée ce jour-là) :

```
export const isoDayOfWeek = (date: Date): number => {
  const day = date.getDay();
  return day === 0 ? 7 : day;
};
```

avec des cas de test couvrant explicitement le rebouclage de semaine (`wraps around to the first day of the next week`) et le programme à un seul jour.

6.3 Inventaire des tests

Fichier	Domaine couvert	Cas de test (ordre de grandeur)
<code>services/api/src/__tests__/compute-daily-calories.test.ts</code>	Mifflin-St Jeor, facteurs d'activité, ajustements d'objectif	18
<code>services/api/src/__tests__/estimate-calories.test.ts</code>	Calories brûlées par séance ($\text{MET} \times \text{poids} \times \text{durée}$)	13
<code>services/api/src/__tests__/program-schedule.test.ts</code>	Jour ISO, jour planifié, prochain jour, rebouclage	10
<code>services/api/src/__tests__/program-schemas.test.ts</code>	Schéma Zod de création/assignation de programme (doublet de jour, bornes 1–7, coercion)	9
<code>services/api/src/__tests__/nutrition-plan-schemas.test.ts</code>	Schéma Zod du plan nutrition (repas, compléments, bornes de cibles)	9
<code>services/api/src/__tests__/shared-schemas.test.ts</code>	Schémas partagés poids/pas/exercice (coercition, bornes)	33
<code>services/api/src/__tests__/invitations.test.ts</code>	Code d'invitation, expiration, statut utilisable	10
<code>services/api/src/__tests__/meal-recipe-request.test.ts</code>	Construction d'une requête IA contrainte par le plan du coach	9
<code>services/api/src/__tests__/health.test.ts</code>	Test de configuration minimal	1

Fichier	Domaine couvert	Cas de test (ordre de grandeur)
<code>services/ai/src/__tests__/effective-tier.test.ts</code>	Palier d'accès effectif élève/lambda	2
<code>services/auth/src/__tests__/health.test.ts</code>	Test de configuration minimal	1

Soit environ **115 cas de test** répartis sur 11 fichiers, exécutés par `vitest run` dans chaque service concerné et agrégés par `pnpm test` (racine) — la même commande que celle bloquante en CI (chapitre 4).

6.4 Limite assumée et axe d'amélioration

Cet inventaire montre une couverture volontairement concentrée là où la valeur métier et le risque de régression sont les plus élevés : calculs (calories), règles de tarification B2B/B2C, planification, contraintes nutritionnelles. Les services `notifications` et `payment` ne disposent pas encore de module `domain/` ni de tests unitaires dédiés : leur logique (mapping des événements Stripe, gabarits d'e-mail) reste aujourd'hui vérifiée manuellement (mode simulé sans clé API, `stripe listen` en local pour les webhooks). De même, le service `auth` ne teste unitairement que sa configuration (test de type *smoke test*), pas encore ses cas d'usage (`authenticate-oauth.usecase.ts`, `refresh-token.usecase.ts`). C'est un axe d'amélioration explicitement identifié — extraire la logique de mapping Stripe et les cas d'usage `auth` vers des modules purs suivant le même pattern que `services/api/src/domain/` — plutôt qu'une lacune non vue.

7. Développer une solution évolutive, sécurisée et accessible — C2.2.3

Compétence couverte : C2.2.3 — Développer une solution logicielle évolutive, en intégrant les recommandations de sécurité et d'accessibilité.

7.1 Évolutivité : un cadre de développement documenté et arbitré

Trois décisions structurent l'évolutivité de la base de code, chacune tracée et justifiée plutôt qu'issue d'une habitude :

Accès direct à PostgreSQL, par service. Le cahier des charges initial suggérait un service dédié aux opérations de base de données. Ce choix a été réexaminé et documenté formellement dans un *Architecture Decision Record* (`docs/adr/0001-acces-bdd-direct-par-service.md`), qui compare trois options — service BDD centralisé, accès direct par service, ORM partagé — et retient l'accès direct : chaque service possède son propre pool pg et ses propres requêtes SQL paramétrées, avec *ownership* des données par domaine (`auth` possède `users`, `api` possède les entités métier, `notifications` possède `notification_logs`). L'ADR assume explicitement l'écart avec l'énoncé initial :

« Créer un service dont le seul rôle est de proxyer des requêtes SQL vers PostgreSQL introduit un couplage fort (...), un goulot d'étranglement (...) et un *Single Point of Failure*. (...) La décision a été documentée, assumée et arbitrée consciemment : ce n'est pas un oubli (...) c'est un choix d'architecture argumenté. » (`docs/adr/0001-acces-bdd-direct-par-service.md`)

L'ADR liste aussi, sans les esquiver, les inconvénients retenus (duplication de configuration de pool, coordination des migrations) et leurs mitigations (centralisation possible dans `@fitapp/shared` si la duplication devient un problème réel, `PgBouncer` en cas de croissance du nombre de connexions) — un **chemin d'évolution** explicite plutôt qu'un choix figé.

Validation partagée entre front et back. Les schémas Zod vivent dans `packages/shared` (`@fitapp/shared`) et sont importés tels quels côté serveur (`safeParse` sur chaque corps de requête) et côté client (formulaires `react-hook-form`). Il n'existe qu'une seule source de vérité pour « qu'est-ce qu'un programme d'entraînement valide » : le schéma `createProgramSchema`, qui borne les jours à 1–7 et interdit un doublon de jour de la semaine par un `.refine()` :

```
export const createProgramSchema = z.object({
  name: z.string().min(1).max(150),
  days: z.array(programDaySchema).max(7).refine(
    (days) => new Set(days.map((d) => d.dayOfWeek)).size === days.length,
    'Un jour de la semaine ne peut apparaître qu\'une fois'
  ),
});
```

Modifier une règle métier ne demande de toucher qu'un fichier, avec effet immédiat aux deux extrémités — et un échec de compilation TypeScript si l'un des deux oublie de s'aligner.

Une hétérogénéité assumée, à harmoniser. Le service `auth` a été structuré très tôt selon un découpage en couches (`domain/`, `application/use-cases/`, `infrastructure/`, `presentation/`), quand les quatre autres services (`api`, `ai`, `notifications`, `payment`) suivent un pattern Fastify plus direct (`routes/`, `services/`, `providers/`, `config/`). Les deux approches sont cohérentes en elles-mêmes, mais l'écart entre les deux styles au sein du même monorepo est un point d'attention identifié

: l'harmonisation vers un unique pattern (probablement le plus simple, `routes/services/providers`, sauf si `auth` gagne en complexité de cas d'usage) est posée comme axe d'évolution plutôt que masquée.

7.2 Sécurité : une déclinaison opérationnelle d'OWASP

Au-delà des mécanismes déjà posés au niveau du prototype (chapitre 5), le tableau suivant relie les pratiques réellement appliquées dans le code aux catégories du OWASP Top 10, cité comme référentiel de veille dès le Bloc 1 :

Catégorie OWASP	Risque visé	Mesure appliquée dans FitCoach AI
A01 — Contrôle d'accès défaillant	Un utilisateur accède à des données ou actions hors de son rôle	<code>authMiddleware + requireRole('coach' \ 'student' \ 'user')</code> sur chaque route sensible ; vérification systématique de <code>request.user.sub</code> avant toute lecture/écriture liée à un utilisateur
A02 — Défaillances cryptographiques	Fuite de secrets ou de mots de passe	Mots de passe hachés (service <code>auth</code>), <code>JWT_SECRET ≥ 32</code> caractères validé par Zod au démarrage, aucun secret committé (<code>.env.gitignored</code> , <code>.env.example</code> sans valeur)
A03 — Injection	Injection SQL	Requêtes 100 % paramétrées (<code>\$1</code> , <code>\$2...</code>) via <code>pg</code> , jamais de concaténation de valeur utilisateur — visible dans chaque route (ex. <code>services/api/src/routes/invitation.routes.ts</code>)
A04 — Conception non sécurisée	Règle métier contournable par construction	Transaction SQL explicite avec verrou de ligne (<code>FOR UPDATE</code>) pour l'acceptation d'une invitation, empêchant une double acceptation concurrente (détail §7.3)
A05 — Mauvaise configuration de sécurité	Services internes exposés par erreur	<code>ai</code> et <code>notifications</code> sans <code>expose/ports</code> dans <code>docker-compose.prod.yml</code> : injoignables même en interne au réseau public ; CORS actuellement ouvert (<code>origin: true</code>) en développement, durcissement vers <code>FRONTEND_URL</code> identifié comme tâche de production (issue #87, cf. chapitre 10, <code>DEPLOYMENT_PROD.md §6</code>)
A06 — Composants vulnérables et obsolètes	CVE dans une dépendance	Montée de version Next.js suite à CVE-2025-29927 (détail chapitre 8) ; <code>pnpm install --frozen-lockfile</code> en CI pour empêcher toute dérive silencieuse
A07 — Identification et authentification défaillantes	Vol/rejeu de jeton	JWT avec <code>issuer/audience</code> vérifiés, expiration configurée (<code>JWT_EXPIRES_IN</code>), OAuth Google en complément du mot de passe
A09 — Carence de journalisation	Incident non détecté	<code>/metrics</code> par service (Prometheus), <code>notification_logs</code> pour tracer les envois — sans jamais loguer la donnée personnelle elle-même (voir ci-dessous)

RGPD et minimisation des données. FitCoach AI manipule des données de santé au sens du RGPD (poids, mensurations, objectifs). La règle appliquée dans le code est stricte : aucune donnée personnelle (email, téléphone, mensurations) n'apparaît en clair dans un `console.log` — seules les erreurs techniques passent par `console.error`, sans les valeurs personnelles associées. Les secrets de service à service (`INTERNAL_API_KEY`) et les jetons ne sont jamais journalisés. C'est une discipline appliquée systématiquement dans les *handlers*, pas une politique déclarative sans traduction technique.

7.3 Un exemple complet de logique sécurisée et transactionnelle

L'acceptation d'une invitation coach→élève illustre la conjonction sécurité + intégrité métier. L'endpoint `POST /invitations/:code/accept` (`services/api/src/routes/invitation.routes.ts`) ouvre une transaction PostgreSQL explicite, verrouille la ligne d'invitation (`FOR UPDATE`) pour empêcher une double acceptation en cas de double clic ou de rejeu, vérifie trois règles métier avant de commiter (invitation utilisable, coach ne rejoignant pas sa propre invitation, élève déjà rattaché à un *autre* coach) et ne promeut en `student` que les comptes `user` (jamais un `coach`) :

```
await client.query('BEGIN');
const invRes = await client.query(
  `SELECT * FROM coach_invitations WHERE code = $1 FOR UPDATE`, [code.toUpperCase()]
);
// ... vérifications (invitation utilisable, pas sa propre invitation, un seul coach actif) ...
await client.query(
  `UPDATE users SET role = 'student' WHERE id = $1 AND role = 'user'`, [userId]
);
await client.query('COMMIT');
```

C'est une **transaction locale simple** au sens de l'ADR-0001 (§ « transactions locales simples ») : pas d'appel réseau inter-service au milieu d'une opération qui doit rester atomique.

7.4 Accessibilité

Les conventions retenues (et appliquées, pas seulement déclarées) : attribut `alt` sur les images, `aria-label` sur les boutons qui ne portent qu'une icône (présent par exemple dans `apps/web/app/coach/page.tsx`, `apps/web/components/session-player.tsx`, `apps/web/app/dashboard/page.tsx`), état de focus visible fourni par les composants `shadcn/ui`, contraste visé au niveau WCAG AA via les tokens du système de design Tailwind, et respect de `prefers-reduced-motion` pour les animations. Ce qui **n'est pas encore fait** est assumé comme tel : aucun audit outillé (Lighthouse, axe-core) n'a encore été formalisé en CI — c'est un contrôle manuel par convention de code, pas un contrôle automatisé. L'ajout d'une passe Lighthouse CI est identifié comme prochaine étape naturelle, cohérente avec le pipeline de qualité déjà en place (chapitre 4).

7.5 Un cycle de développement complet, de bout en bout

Pour illustrer concrètement le processus de développement (et pas seulement l'énoncer), voici le cycle réel le plus récent au moment de la rédaction de ce dossier : **issue #103 « enregistrer une recette »**.

Le besoin. Le Frigo Mode (autonome) et la nutrition contrainte (élève) génèrent des recettes par IA, mais rien ne permettait de les conserver : fermer l'écran perdait la recette. Le service `api` exposait déjà, de longue date, les routes de persistance (`GET/POST/DELETE /api/recipes`, table `recipes` présente dans `scripts/init-db.sql` depuis la V1 mobile) — mais aucune interface web ne les consommait.

Le développement. Branche `feature/save-recipe`, ouverte depuis `dev` :

- Un bouton « Enregistrer la recette » ajouté à `apps/web/components/recipe-card.tsx`, actif uniquement quand une source est connue (`saveSource: "frigo" | "coach-plan"`), avec trois états (`idle` / `saving` / `saved`) et gestion d'erreur par `toast` ;
- Une nouvelle page `apps/web/app/recipes/page.tsx` (liste paginée « Charger plus », suppression avec confirmation via un composant `Dialog` — jamais de `confirm()` natif, conformément à la convention de composants retenue) ;
- Le client typé `recipeApi` (`apps/web/lib/api.ts`) ajouté pour `save/list/remove`, s'appuyant sur le schéma partagé déjà défini côté back :

```
export const saveRecipeSchema = z.object({
  title: z.string().min(1).max(255),
  ingredients: z.array(ingredientInputSchema).min(1).max(50),
  instructions: z.array(z.string().max(1000)).min(1).max(50),
  isFromFrigoMode: z.boolean().optional(),
  // ...
});
```

- Le serveur revalide ce même schéma avant toute écriture (`saveRecipeSchema.safeParse(request.body)`), sans jamais faire confiance à ce que le front a déjà validé — la règle énoncée au §7.2 appliquée littéralement.

L'intégration. `pnpm typecheck && pnpm lint && pnpm build && pnpm test` en local, ouverture de la **pull request #105** vers `dev` (« Closes #103 »), CI verte (`ci.yml`, chapitre 4), auto-revue documentée dans la description de PR (gabarit `.github/pull_request_template.md`), **squash merge**. Le merge sur `main` déclenche automatiquement le redéploiement Vercel du front (chapitre 8) — aucune étape manuelle après la fusion.

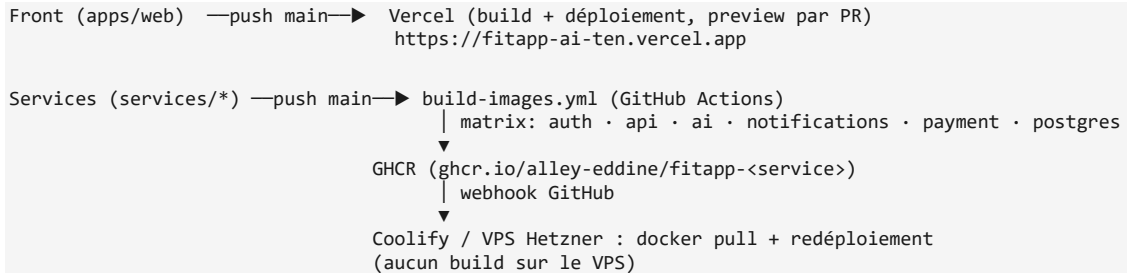
Ce cycle — petit par sa taille, complet par sa forme — est représentatif du fonctionnement courant du projet : une issue, une branche `feature/*`, un développement qui réutilise l'existant plutôt que de le dupliquer (schéma déjà partagé, routes déjà exposées), une CI qui protège la fusion, un déploiement qui suit sans intervention.

8. Déploiement continu — C2.2.4

Compétence couverte : C2.2.4 — Déployer en continu une solution logicielle en configurant un système d'intégration/déploiement continu.

8.1 Vue d'ensemble de la chaîne

Le déploiement est scindé en deux circuits indépendants, chacun automatique :



Le workflow `.github/workflows/build-images.yml` ne se déclenche que sur push vers `main`, et seulement si les chemins `services/**`, `packages/shared/**` ou `pnpm-lock.yaml` sont concernés (plus un déclenchement manuel `workflow_dispatch`) — un filtre de chemin qui évite de reconstruire six images Docker pour une modification qui ne touche que le front. Il construit en parallèle (matrice) les cinq microservices **et** une sixième image, `postgres`, qui embarque le schéma applicatif (voir §8.4), et les pousse vers GHCR avec deux étiquettes (`:latest` et `:<sha>` du commit) :

```

strategy:
  matrix:
    service: [auth, api, ai, notifications, payment, postgres]
steps:
  - uses: docker/build-push-action@v6
    with:
      file: services/${{ matrix.service }}/Dockerfile
      push: true
      tags: |
        ghcr.io/alley-eddine/fitapp-${{ matrix.service }}:latest
        ghcr.io/alley-eddine/fitapp-${{ matrix.service }}:${{ github.sha }}
      cache-from: type=gha
      cache-to: type=gha,mode=max
  
```

Le cache GitHub Actions (`type=gha`) est réutilisé d'un build à l'autre : les étapes `pnpm install` et de compilation qui n'ont pas changé ne sont pas rejouées — un gain de temps et de calcul, dans la continuité de la logique de sobriété déjà retenue pour la CI (chapitre 4) et pour le choix même de déporter le build hors du VPS (§3.3).

Le front, lui, est branché directement sur Vercel : chaque push sur `main` déclenche un build et un déploiement de production ; chaque *pull request* obtient automatiquement un déploiement de prévisualisation isolé.

8.2 Le VPS ne construit rien

`docker-compose.prod.yml` ne référence que des images déjà construites (`image: ghcr.io/alley-eddine/fitapp-api:latest`, etc.) : Coolify les télécharge et démarre les conteneurs, il ne lance jamais `docker build`. C'est la correction structurelle de l'incident de saturation mémoire décrit au §3.3 — la cause étant éliminée, pas seulement compensée.

8.3 Étanchéité public/privé au niveau du déploiement

`auth`, `api` et `payment` déclarent un `expose` (port visible pour le routeur interne Traefik de Coolify, qui leur attribue un nom de domaine `sslip.io` et un certificat TLS) :

```
auth:
  image: ghcr.io/alley-eddine/fitapp-auth:latest
  expose:
    - "3001"
  depends_on:
    postgres:
      condition: service_healthy
```

`ai` et `notifications` ne déclarent ni `expose` ni `ports` : ils ne sont routables que par leur nom de service DNS interne au réseau Docker de Coolify (`http://ai:3003`, `http://notifications:3004`), jamais par une URL publique. C'est la même logique de moindre surface d'attaque qu'au niveau applicatif (chapitre 7), reportée au niveau infrastructure.

8.4 Incidents de mise en production, diagnostic et résolution

Le passage en production a fait apparaître plusieurs incidents réels, chacun diagnostiqué puis corrigé — la liste qui suit est aussi la mémoire technique du projet, au sens où chaque ligne correspond à une branche ou un commit identifiable.

Incident	Symptôme observé	Diagnostic	Correction
Ports non exposés	Traefik (routeur de Coolify) répondait « no available server » sur les URLs <code>sslip.io</code> fraîchement générées	Les services <code>auth/api/payment</code> ne déclaraient pas leur port dans le fichier <code>Compose</code> de production ; Traefik ne savait vers quel port interne router le domaine	Ajout de <code>expose</code> : <code>["3001"/"3002"/"3005"]</code> sur les trois services publics — commit <code>fix(deploy): expose service ports so Coolify routes the domains</code> , fusionné via la PR #96 (branche <code>hotfix/coolify-expose-ports</code>)

Incident	Symptôme observé	Diagnostic	Correction
Schéma PostgreSQL jamais rejoué	Les services démarraient, mais toute requête SQL échouait (tables absentes)	Le schéma était injecté par un <i>bind-mount</i> du fichier <code>scripts/init-db.sql</code> (comme en développement), or Coolify ne conserve pas les fichiers du dépôt sur le volume au moment de l'exécution — seul un volume Docker nommé persiste	Le schéma est désormais embarqué dans l'image postgres elle-même (<code>services/postgres/Dockerfile</code>), exécuté automatiquement par l'image officielle au premier démarrage sur un volume vide : <code>COPY scripts/init-db.sql /docker-entrypoint-initdb.d/init-db.sql</code> . Fusionné via la PR #94 (branche <code>hotfix/postgres-baked-schema</code>). Pour une base déjà peuplée par un schéma partiel, la procédure documentée consiste à rejouer manuellement les migrations idempotentes (<code>scripts/migrations/*.sql</code>) avec <code>psql -f</code>
Variables d'environnement manquantes	Un service démarrait puis s'arrêtait immédiatement (redémarrages en boucle)	Le schéma Zod de <code>config/env.ts</code> (§3.1) rejette toute variable absente ou invalide et appelle <code>process.exit(1)</code> — comportement voulu, mais dont la cause n'était pas toujours évidente à relier à une variable précise absente du panneau Coolify	Vérification exhaustive de chaque variable du service face à <code>.env.production.example</code> , qui documente désormais la liste complète attendue par variable et par service

Incident	Symptôme observé	Diagnostic	Correction
Blocage du front — CVE Next.js	Un contrôle de sécurité automatisé (Vercel) empêchait la mise en production	Version 15.1.6 de Next.js concernée par la CVE-2025-29927 (score CVSS 9.1) : un en-tête <code>x-middleware-subrequest</code> forgé permet de contourner entièrement le <i>middleware</i> , donc les vérifications d'autorisation qu'il porte, sur les versions antérieures à 15.2.3	Montée de version vers 15.5.21 (<code>apps/web/package.json</code>), très au-delà du correctif minimal — la CI (<code>typecheck/lint/build</code>) a validé la non-régression avant la mise en production
HTTPS sans nom de domaine possédé	Besoin d'un accès public chiffré sans budget ni délai d'achat de domaine	—	Coolify génère des sous-domaines <code>sslip.io</code> qui résolvent directement vers l'IP du VPS (<code><service>.<IP>.sslip.io</code>), combinés à Traefik + Let's Encrypt pour un certificat TLS automatique. Documenté comme solution transitoire dans <code>DEPLOYMENT_PROD.md</code> , avec un chemin d'évolution explicite vers un nom de domaine acheté sans changement de code (seules les variables d'environnement d'URL seraient à mettre à jour)
Saturation mémoire du build (OOM)	VPS figé pendant un déploiement multi-services	cf. §3.3 (diagnostic détaillé)	Build déporté vers GitHub Actions + GHCR (§8.1), <i>swapfile</i> de 4 Go en filet de sécurité

Chacune de ces corrections est délimitée par une branche nommée explicitement selon la convention retenue (`hotfix/*` pour un correctif urgent de production, cf. §8.5), ce qui rend l'historique Git lui-même consultable comme un second journal d'incidents, en plus du suivi par issues GitHub.

8.5 Discipline Git derrière le déploiement continu

Le modèle de branches (`main` déployable ↔ `dev` d'intégration ↔ `feature/*` / `fix/*` / `hotfix/*`) et les *conventional commits* anglais (`feat`, `fix`, `chore`, `refactor`, `docs`, `test`, `perf`, `style`, `ci`) sont appliqués sur l'ensemble de l'historique récent, avec fusion en **squash merge** systématique pour garder un historique de `dev/main` lisible — visible par exemple sur les fusions récentes :

```

Merge pull request #102 from Alley-eddine/feature/student-start-any-session
feat(student): make every program session startable, not just today's
Merge pull request #96 from Alley-eddine/hotfix/coolify-expose-ports
fix(deploy): expose service ports so Coolify routes the domains
Merge pull request #94 from Alley-eddine/hotfix/postgres-baked-schema

```

Un tag `v1.0.0` marque la première version jugée stable de la plateforme, cohérent avec le numéro de version `1.0.0` porté par `package.json` à la racine et dans chaque service/application du monorepo. Le suivi détaillé, versionné et daté de chaque publication (journal des versions) relève du Bloc 4 ; ce tag en est le point de départ.

8.6 Indicateurs de déploiement

Indicateur	Constat actuel
Fréquence de déploiement du front	À chaque push sur <code>main</code> (automatique, Vercel)
Fréquence de déploiement du back	À chaque push sur <code>main</code> touchant <code>services/**/packages/shared/**</code> , plus déclenchement manuel possible
Délai entre merge et mise en production (front)	De l'ordre de la minute (build Vercel)
Délai entre merge et mise en production (back)	Durée du build matriciel GitHub Actions + webhook Coolify (quelques minutes), sans intervention manuelle une fois les images publiques sur GHCR
Incidents de déploiement recensés et corrigés	6 (tableau §8.4), tous avec cause racine identifiée et correctif tracé par PR/branche

9. Cahier de recettes — C2.3.1

Compétence couverte : C2.3.1 — Élaborer un cahier de recettes en vue de la présentation au client des fonctionnalités développées.

9.1 Méthode

Le cahier de recettes ci-dessous est exécuté manuellement sur l'environnement de production (front <https://fitapp-ai-ten.vercel.app>, back sur les URLs `sslip.io` récapitulées en tête de dossier), à l'aide des deux comptes de démonstration (`markus.demo@fitcoach.local` côté coach, `emma.demo@fitcoach.local` côté élève, déjà rattachée). Chaque scénario porte un identifiant (TC-), le résultat attendu, le résultat constaté et un statut. Les statuts KO* ne sont pas des échecs cachés : ils alimentent directement le plan de correction des bugs (chapitre 10), avec renvoi vers le numéro d'issue GitHub correspondant.

9.2 Scénarios — parcours autonome

#	Scénario	Résultat attendu	Constat	Statut
TC-01	Inscription d'un compte <code>user</code> , saisie du profil (âge, poids, taille, objectif)	Le besoin calorique quotidien est calculé (Mifflin-St Jeor) et affiché	Conforme	OK
TC-02	Connexion, accès au tableau de bord	Jeton JWT émis, accès aux routes protégées, refus (401) sans jeton	Conforme	OK
TC-03	Frigo Mode : saisie d'ingrédients disponibles, génération d'une recette IA	Recette cohérente avec les ingrédients et le profil, sans contrainte de coach (lambda)	Conforme	OK
TC-04	Enregistrer une recette générée, la retrouver dans « Mes recettes », la supprimer	Recette persistée (POST <code>/api/recipes</code>), listée avec pagination, supprimable avec confirmation	Conforme depuis la PR #105	OK (anciennement #103)

9.3 Scénarios — parcours coach

#	Scénario	Résultat attendu	Constat	Statut
TC-05	Création d'un programme d'entraînement (7 jours max, un seul bloc par jour de semaine)	Refus d'un doublon de jour ; refus au-delà de 7 jours	Conforme (<code>createProgramSchema</code>)	OK
TC-06	Création d'un plan nutrition (repas par kcal/macros, aliments, compléments)	Plan enregistré, versionné par phase	Conforme	OK

#	Scénario	Résultat attendu	Constat	Statut
TC-07	Génération d'un lien d'invitation pour un élève	Code à 8 caractères non ambigu, expiration à 14 jours	Conforme (generateInvitationCode, INVITATION_TTL_DAYS)	OK
TC-08	Suivi de la progression d'un élève rattaché (séances faites, poids, mensurations)	Le coach visualise l'adhérence et les courbes de l'élève	Conforme	OK
TC-09	Un compte fraîchement promu coach ouvre /onboarding	Redirection directe vers le tableau de bord coach, sans paywall B2C	Le paywall B2C s'affiche : seul le rôle student est exclu du garde d'onboarding, pas le rôle coach	KO — #98

9.4 Scénarios — parcours élève

#	Scénario	Résultat attendu	Constat	Statut
TC-10	Élève rattaché : consultation du programme assigné	Programme affiché avec la phase en cours	Conforme	OK
TC-11	Lancer une séance planifiée un jour différent du jour prévu (rattrapage)	La séance reste démarrable n'importe quel jour de la semaine	Conforme depuis la PR #102 — le composant affiche un bouton « Démarrer » sur chaque jour du programme, pas seulement sur le jour du calendrier	OK (anciennement #101)
TC-12	Élève rattaché : jamais de paywall B2C, même après changement de palier B2C sous-jacent	Accès toujours au niveau premium effectif tant que le rattachement au coach est actif	Conforme (resolveEffectiveTier, vérifié côté serveur dans ai.routes.ts)	OK
TC-13	Élève clique un lien d'invitation alors qu'il n'est pas connecté, doit s'inscrire/se connecter puis rejoindre le coach sans nouvelle manipulation	Retour automatique sur l'écran d'invitation après authentification, rattachement immédiat	router.push de la page de connexion redirige systématiquement vers /onboarding ou /dashboard, sans mémoriser le code d'invitation d'origine : l'utilisateur doit rouvrir le lien une seconde fois après s'être authentifié	KO — #99

#	Scénario	Résultat attendu	Constat	Statut
TC-14	Génération d'une recette IA à partir d'un repas imposé par le coach	Recette dans le cadre kcal/macros/aliments du repas, ingrédients du repas complétés par ceux de l'élève	Conforme (<code>buildMealRecipeRequest</code>)	OK
TC-15	Coach consulte un jour de repos dans le générateur de programme et y attache une instruction spécifique (repos actif, message)	Le jour de repos est une entité éditable au même titre qu'un jour d'entraînement	Un jour de repos n'existe qu'en creux (absence de bloc pour ce jour) ; aucune instruction ni message dédié n'est attachable	KO — #100
TC-16	Repas du plan nutrition avec plusieurs options interchangeables (format réel utilisé par Markus, ex. « Repas 1 : option A ou option B »)	Le plan nutrition supporte plusieurs options par repas	La table <code>nutrition_meals</code> et le schéma <code>nutritionMealSchema</code> ne portent qu'un seul jeu de cibles/aliments par repas : aucune option alternative n'est modélisable	KO — #104

9.5 Scénarios transverses (sécurité, notifications)

#	Scénario	Résultat attendu	Constat	Statut
TC-17	Appel direct à une route protégée sans jeton, puis avec un rôle insuffisant	401 sans jeton, 403 avec un rôle non autorisé	Conforme (<code>authMiddleware</code> , <code>requireRole</code>)	OK
TC-18	Appel du service notifications sans <code>x-internal-key</code>	401	Conforme (<code>internalAuth</code>)	OK
TC-19	Paiement B2C (Stripe, mode test) — souscription, webhook, facture	Abonnement activé, facture envoyée	Conforme (mode test)	OK
TC-20	Vérification email et réinitialisation de mot de passe par SMS	E-mail/SMS envoyés (ou simulés si clé API absente)	Conforme — mode simulé en développement, réel en production (Resend Pro / Twilio)	OK

#	Scénario	Résultat attendu	Constat	Statut
TC-21	Génération de recette IA avec le fournisseur externe indisponible (résilience)	Message d'erreur maîtrisé et reprise possible côté utilisateur	L'erreur est propre mais la fonctionnalité reste indisponible tant que le fournisseur unique est hors service	KO — #97 (détail chapitre 10)

9.6 Synthèse du cahier de recettes

Sur 21 scénarios exécutés, **16 sont conformes (OK)** et **5 révèlent un écart consigné (KO)**, chacun tracé par un numéro d'issue repris intégralement au chapitre suivant. Cette proportion (76 % de conformité sur un cahier de recettes qui couvre volontairement les trois rôles et les cas limites, pas seulement le chemin heureux) est jugée saine à ce stade : les écarts identifiés portent sur des raffinements d'expérience (redirection après connexion, richesse du modèle de repas, jour de repos éditorialisé, résilience de la dépendance IA) et non sur des failles de sécurité ou des pertes de données.

10. Plan de correction des bugs — C2.3.2

Compétence couverte : C2.3.2 — Élaborer un plan de correction des bugs afin d'améliorer la qualité du produit.

10.1 Processus de gestion des anomalies

Toute anomalie constatée (cahier de recettes, usage réel par Markus et Emma, ou revue de code) est consignée en **issue GitHub**, avec un label de nature (bug, feature, chore), un label de priorité (priority:high|med|low) et rattachée à un tableau Kanban (Backlog → Todo → In progress → Review → Done). La correction se fait sur une branche `fix/*` (non urgente) ou `hotfix/*` (urgente, production), la *pull request* correspondante référence l'issue par la formule `Closes #<n>` du gabarit de PR, et la CI (chapitre 4) doit être verte avant fusion. Ce processus est le même que celui déjà illustré au chapitre 8 pour les incidents de déploiement, et au chapitre 7 (§7.5) pour le développement de fonctionnalité.

10.2 Anomalies résolues (preuve par le code et l'historique Git)

Issue	Symptôme	Cause	Correction	Preuve
#82	Le paywall B2C pouvait apparaître à un élève fraîchement rattaché à son coach	Rôle « student » appliqué trop tard après l'acceptation de l'invitation : l'onboarding B2C se déclenchait sur un rôle en cache obsolète	Application immédiate du rôle élève dès le rattachement au coach, avant toute redirection	Branche <code>fix/student-paywall-onboarding-role</code> , PR #85 , commit <code>fix(web): apply student role right after joining a coach</code>
#101	L'élève ne pouvait démarrer que la séance du jour calendaire ; impossible de rattraper une séance manquée un autre jour	La vue programme ne proposait un bouton « Démarrer » que pour le jour correspondant à la date du jour	Toute séance du programme devient démarrable, quel que soit le jour ; la liste hebdomadaire affiche un bouton « Démarrer » sur chaque ligne	Branche <code>feature/student-start-any-session</code> , PR #102 , commit <code>feat(student): make every program session startable, not just today's ; commentaire de code explicite dans apps/web/app/student/program/page.tsx : « every session is startable; the student spaces them freely »</code>

Issue	Symptôme	Cause	Correction	Preuve
#103	Aucune recette générée par l'IA ne pouvait être conservée	Les routes de persistance existaient côté service <code>api</code> mais n'étaient consommées par aucune interface web	Bouton « Enregistrer la recette », page « Mes recettes » (liste, suppression confirmée), client <code>recipeApi</code> typé	Branche <code>feature/save-recipe</code> , PR #105 (détail complet §7.5)

10.3 Anomalies ouvertes — détaillées

Issue	Symptôme constaté (cahier de recettes)	Cause racine identifiée	Plan de correction	Priorité
#98	Un compte coach peut voir l'écran d'onboarding B2C (paiement)	<code>apps/web/app/onboarding/page.tsx</code> n'exclut du chargement des offres B2C que <code>role === "student"</code> ; le rôle <code>coach</code> n'est pas testé	Ajouter la même exclusion pour <code>role === "coach"</code> (redirection directe vers <code>/coach</code>), avec un test manuel de non-régression sur le parcours élève existant	Haute — expérience commerciale lui-même
#99	Un lambda non connecté qui clique un lien d'invitation doit se réauthentifier puis rouvrir le lien une seconde fois	<code>apps/web/app/login/page.tsx</code> redirige systématiquement vers <code>/onboarding</code> ou <code>/dashboard</code> après authentification, sans mémoriser l'origine (<code>/join/[code]</code>)	Propager le code d'invitation en paramètre de redirection à travers <code>/login</code> (ex. <code>?redirect=/join/<code></code>), puis rediriger vers cette cible après connexion/inscription au lieu d'une destination fixe	Haute — d'entrée principale nouveaux élèves
#100	Impossible pour le coach d'attacher une instruction à un jour de repos dans le générateur de programme	Le modèle actuel (<code>training_program_days</code> / composant <code>apps/web/app/coach/programs/[id]/page.tsx</code>) ne représente que les jours actifs ; un jour de repos n'est qu'une absence, sans ligne ni champ associé	Faire du jour de repos une entité explicite du programme (<code>isRestDay</code> avec note optionnelle), affichée dans le générateur au même titre qu'un jour d'entraînement, et reprise par le gabarit de notification « jour de repos » (déjà prévu au Bloc 1)	Moyenne

Issue	Symptôme constaté (cahier de recettes)	Cause racine identifiée	Plan de correction	Priorité
#104	Le plan nutrition ne peut pas représenter le format réel utilisé par Markus (un repas avec plusieurs options interchangeables)	<code>nutrition_meals</code> (un seul jeu de cibles/aliments par ligne) et <code>nutritionMealSchema</code> ne portent pas de notion d'option	Introduire une sous-entité « option de repas » (0..n options par repas, chacune avec sa propre liste d'aliments), avec migration idempotente dédiée (<code>scripts/migrations/</code>) et mise à jour du schéma partagé	Moyenn bloque l fidélité a support du coach
#97	La génération de recettes dépend d'un fournisseur IA unique (Groq) : en cas d'indisponibilité du fournisseur, la fonctionnalité est hors service	Absence de fournisseur de repli — dépendance externe unique	Introduire un fournisseur de secours derrière la même interface de génération (l'appel est déjà isolé côté service <code>ai</code>), avec bascule automatique en cas d'erreur ou de délai dépassé	Amélior — résilie

L'issue #97 n'est pas une anomalie fonctionnelle mais une **demande d'amélioration de résilience** consignée dans le même tableau de suivi : la traiter dans le plan de correction reflète la réalité d'un backlog unique où anomalies et améliorations coexistent et sont priorisées ensemble.

10.4 Priorisation

La priorisation suit deux axes — impact sur l'expérience du commanditaire et de ses élèves, effort de correction — cohérents avec la matrice de risques déjà posée au Bloc 1 :

Priorité	Issues	Logique
Haute	#98, #99	Impact direct sur le premier contact (coach fraîchement créé, élève rejoignant son coach) — chemin critique du produit
Moyenne	#100, #104	Impact sur la fidélité au besoin réel du coach (richesse du contenu), pas sur la sécurité ou l'accès
Amélioration	#97	Résilience de la dépendance au fournisseur IA — planifiée après les correctifs de priorité haute

10.5 Indicateurs de suivi des anomalies

Indicateur	Valeur constatée
Anomalies consignées à date	8 (#82, #97, #98, #99, #100, #101, #103, #104)

Indicateur	Valeur constatée
Anomalies résolues et déployées	3 (#82, #101, #103)
Anomalies ouvertes	5, dont une amélioration de résilience (#97)
Anomalies ouvertes de priorité haute	2 (#98, #99)
Délai observé issue → PR de correction (cas #82, #101, #103)	Un seul cycle de développement (branche unique, pas de retour en arrière)

11. Documentation technique : déploiement et utilisateur — C2.4.1

Compétence couverte : C2.4.1 — Élaborer les documents techniques destinés aux équipes projet et aux utilisateurs.

11.1 Documentation de déploiement

`DEPLOYMENT_PROD.md`, à la racine du dépôt, est le runbook de référence pour mettre en production ou reconstituer l'environnement : schéma d'ensemble (Vercel + Coolify/Hetzner), ordre de déploiement en trois temps (back d'abord pour obtenir les URLs `sslip.io`, puis front pour obtenir l'URL Vercel, puis retour au back pour renseigner les URLs croisées), génération des secrets (`openssl rand -hex 32` pour `JWT_SECRET`, etc.), callbacks externes à mettre à jour (Google OAuth, webhook Stripe, domaine Resend), commandes de vérification post-déploiement (`curl .../health` sur les trois services publics) et parcours de non-régression fonctionnelle à rejouer après chaque mise en production majeure. Le fichier `.env.production.example` documente, variable par variable, tout ce qu'attend `docker-compose.prod.yml`, sans jamais porter de valeur réelle.

11.2 Documentation de décision architecturale

`docs/adr/0001-acces-bdd-direct-par-service.md` documente, au format *Architecture Decision Record* (contexte, options envisagées, décision, conséquences positives et négatives avec mitigations), l'écart assumé entre le cahier des charges initial et l'architecture retenue pour l'accès aux données (chapitre 7, §7.1). C'est un format volontairement choisi pour qu'une décision technique reste compréhensible et justifiable des mois plus tard, y compris par quelqu'un qui n'a pas suivi le projet au jour le jour — un autre développeur qui rejoindrait le projet, ou le jury de certification.

11.3 Documentation portée par le code lui-même

Une partie significative de la documentation technique de FitCoach AI n'est pas séparée du code : elle est le commentaire qui explique le **pourquoi** d'un choix, directement au-dessus de la fonction concernée — convention appliquée dans tous les modules `domain/` cités au chapitre 6 (par exemple le commentaire au-dessus de `resolveEffectiveTier` qui explique la règle B2B avant même de lire l'implémentation d'une ligne), dans le gabarit de *pull request* (`.github/pull_request_template.md`, qui impose d'expliciter le contexte et le plan de test), et dans les fichiers de configuration eux-mêmes (`docker-compose.prod.yml` et `services/postgres/Dockerfile` commentent explicitement *pourquoi* le schéma est embarqué dans l'image plutôt que monté en volume).

11.4 Documentation utilisateur : état actuel et plan

À ce stade, la documentation destinée à l'utilisateur final (coach, élève, autonome) est portée **par le produit lui-même** plutôt que par un document séparé : écran d'onboarding qui explique les fonctionnalités au premier lancement (`apps/web/app/onboarding/page.tsx`), libellés et messages d'erreur en français clair sur chaque formulaire, écran de bienvenue lors de l'acceptation d'une invitation qui explicite ce que l'élève doit attendre (« Rejoins son espace : tu recevras tes programmes et ton plan nutrition, déjà réglés. Rien à configurer. », `apps/web/app/join/[code]/page.tsx`). C'est cohérent avec l'objectif produit posé au Bloc 1 : minimiser ce que l'utilisateur doit apprendre pour utiliser l'application.

Un guide utilisateur autonome (à destination de Markus, pour qu'il puisse aussi former de nouveaux élèves sans intervention du développeur) n'existe pas encore sous forme de document dédié : c'est identifié comme le prochain livrable de documentation à produire, sous la forme d'un support court par rôle (coach / élève), complémentaire à ce que l'interface explique déjà.

12. Synthèse et chemin d'évolution

Ce dossier a montré, compétence par compétence, comment FitCoach AI est passé du cadrage (Bloc 1) à une plateforme réellement déployée, avec des environnements de développement, de test et de production séparés et surveillés (C2.1.1), une intégration continue qui protège chaque fusion (C2.1.2), un prototype où la sécurité est structurelle dès l'amorçage (C2.2.1), une suite de tests unitaires concentrée sur la logique métier à plus forte valeur et à plus fort risque (C2.2.2), une base de code pensée pour évoluer et documentée dans ses arbitrages (C2.2.3), une chaîne de déploiement continu qui a traversé et corrigé des incidents réels de production (C2.2.4), un cahier de recettes qui a réellement produit des anomalies exploitables (C2.3.1), un plan de correction qui les priorise et les trace jusqu'au code (C2.3.2), et une documentation technique qui vit pour partie dans le code lui-même (C2.4.1).

Les axes d'évolution identifiés dans ce dossier — harmonisation du pattern d'architecture entre services (§7.1), extraction de la logique métier `payment/notifications` vers des modules testables (§6.4), correction des anomalies #98/#99/#100/#104 (chapitre 10), durcissement du CORS en production (§7.2, issue #87), audit d'accessibilité outillé (§7.4), guide utilisateur dédié (§11.4) — ne sont volontairement pas approfondis ici : ils relèvent du **maintien en condition opérationnelle** (Bloc 4), qui prend appui sur la chaîne CI/CD documentée dans ce dossier pour livrer ses correctifs. Le pilotage de ces priorités dans le temps (planification, arbitrages, comptes rendus au commanditaire) est le sujet du Bloc 3.

13. Annexes

13.1 Artefacts publics cités dans ce dossier

Artefact	Rôle
DEPLOYMENT_PROD.md	Runbook de mise en production
.env.production.example	Documentation exhaustive des variables d'environnement de production
docs/adr/0001-acces-bdd-direct-par-service.md	Décision d'architecture (accès BDD par service)
.github/workflows/ci.yml	Intégration continue (Lint · Typecheck · Test · Knip · Build)
.github/workflows/build-images.yml	Construction et publication des images Docker (GHCR)
.github/pull_request_template.md	Gabarit de pull request
docker-compose.yml / docker-compose.prod.yml	Environnements Docker développement / production
scripts/init-db.sql, scripts/migrations/*.sql	Schéma de base et migrations idempotentes
knip.config.ts	Détection de code et dépendances mortes (monorepo)
packages/shared/src/schemas/*	Schémas Zod partagés front/back
services/*/src/domain/*	Logique métier pure et testée unitairement
docs/rncp/bloc1/Bloc1_Deck.md, Bloc1_Script_Soutenance.md	Cadrage (Bloc 1), référencé pour continuité

13.2 Comptes de démonstration

Compte	Rôle	Usage
markus.demo@fitcoach.local	coach	Programmes, plans nutrition, suivi d'élève
emma.demo@fitcoach.local	student	Élève rattachée, exécution zéro-configuration

13.3 Issues et pull requests citées

#82, #85, #94, #96, #97, #98, #99, #100, #101, #102, #103, #104, #105 — GitHub, dépôt du projet FitCoach AI.

13.4 Glossaire

- **PWA** : *Progressive Web App*, application web installable avec notifications push et fonctionnement hors-ligne partiel.
- **JWT** : *JSON Web Token*, jeton d'authentification signé.

- **RBAC** : *Role-Based Access Control*, contrôle d'accès fondé sur le rôle.
- **GHCR** : *GitHub Container Registry*, registre d'images Docker.
- **ADR** : *Architecture Decision Record*, document de décision d'architecture.
- **OOM** : *Out Of Memory*, saturation de la mémoire disponible.
- **CVE** : *Common Vulnerabilities and Exposures*, identifiant public de vulnérabilité logicielle.